

```
import os
import io
import sys
import json
import requests
from dotenv import load_dotenv
from openai import OpenAI
import google.generativeai
import anthropic
from IPython.display import Markdown, display, update_display
import gradio as gr
import subprocess
```

Mục đích: Nhập các thư viện cần thiết để làm việc với mô hình AI, hiển thị nội dung, và tạo giao diện tương tác.

Một số thư viện quan trọng:

dotenv: Quản lý biến môi trường (API key).

openai, anthropic, google.generativeai: Các thư viện để gọi mô hình AI.

gradio: Tạo giao diện web đơn giản cho mô hình.

```
load_dotenv(override=True)
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')
os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env')
os.environ['HF_TOKEN'] = os.getenv('HF_TOKEN', 'your-key-if-not-using-env')
```

Mục đích:

Tải API keys từ file .env để tránh lưu trữ trực tiếp trong mã nguồn.

Thiết lập biến môi trường cho OpenAI, Claude (Anthropic), và Hugging Face.

Nếu không có API key trong .env, sẽ dùng chuỗi mặc định ('your-key-if-not-using-env').

```
openai = OpenAI()
claude = anthropic.Anthropic()
OPENAI_MODEL = "gpt-4o"
CLAUDE_MODEL = "claude-3-5-sonnet-20240620"
```

Mục đích:

Khởi tạo đối tượng openai để gọi mô hình GPT-4o.

Khởi tạo đối tượng claude để gọi mô hình Claude-3.5 Sonnet.

Xác định mô hình AI sẽ sử dụng.

```
system_message = "You are an assistant that reimplements Python code in high performance C++  
system_message += "Respond only with C++ code; use comments sparingly and do not provide any C  
system_message += "The C++ response needs to produce an identical output in the fastest possibl
```

Mục đích:

Chuẩn bị danh sách thông điệp để gửi đến AI.

system_message: Thiết lập quy tắc chung cho AI.

user_prompt_for(python): Chứa đoạn mã Python cần chuyển đổi.

```
def user_prompt_for(python):  
    user_prompt = "Rewrite this Python code in C++ with the fastest possible implementation th  
    user_prompt += "Respond only with C++ code; do not explain your work other than a few com  
    user_prompt += "Pay attention to number types to ensure no int overflows. Remember to #in  
    user_prompt += python  
    return user_prompt
```

Mục đích:

Chuẩn bị danh sách thông điệp để gửi đến AI.

system_message: Thiết lập quy tắc chung cho AI.

user_prompt_for(python): Chứa đoạn mã Python cần chuyển đổi.

```
def messages_for(python):
    return [
        {"role": "system", "content": system_message},
        {"role": "user", "content": user_prompt_for(python)}
    ]
```

Mục đích:

Chuẩn bị danh sách thông điệp để gửi đến AI.

system_message: Thiết lập quy tắc chung cho AI.

user_prompt_for(python): Chứa đoạn mã Python cần chuyển đổi.

```
# write to a file called optimized.cpp

def write_output(cpp):
    code = cpp.replace("```cpp", "").replace("```", "")
    with open("optimized.cpp", "w") as f:
        f.write(code)
```

Mục đích:

Hàm này lưu đoạn mã C++ tối ưu hóa vào tệp optimized.cpp.

Trước khi ghi, nó loại bỏ các dấu Markdown (```cpp và ```) để đảm bảo chỉ có mã nguồn.

```
def optimize_gpt(python):
    stream = openai.chat.completions.create(model=OPENAI_MODEL, messages=messages_for(python))
    reply = ""
    for chunk in stream:
        fragment = chunk.choices[0].delta.content or ""
        reply += fragment
        print(fragment, end='', flush=True)
    write_output(reply)
```

Mục đích:

Gửi đoạn mã Python đến mô hình **GPT-4o** để dịch sang C++.

Sử dụng streaming để nhận kết quả theo từng phần nhỏ thay vì chờ toàn bộ phản hồi.

Ghi mã C++ nhận được vào tệp `optimized.cpp`.

```
def optimize_claude(python):
    result = claude.messages.stream(
        model=CLAUDE_MODEL,
        max_tokens=2000,
        system=system_message,
        messages=[{"role": "user", "content": user_prompt_for(python)}],
    )
    reply = ""
    with result as stream:
        for text in stream.text_stream:
            reply += text
            print(text, end="", flush=True)
    write_output(reply)
```

Mục đích:

Gửi mã Python đến **Claude-3.5 Sonnet** để dịch sang C++.

Nhận kết quả dần dần thông qua **streaming**.

Ghi mã C++ nhận được vào `optimized.cpp`.

```

python_hard = """# Be careful to support large number sizes

def lcg(seed, a=1664525, c=1013904223, m=2**32):
    value = seed
    while True:
        value = (a * value + c) % m
        yield value

def max_subarray_sum(n, seed, min_val, max_val):
    lcg_gen = lcg(seed)
    random_numbers = [next(lcg_gen) % (max_val - min_val + 1) + min_val for _ in range(n)]
    max_sum = float('-inf')
    for i in range(n):
        current_sum = 0
        for j in range(1, n):
            current_sum += random_numbers[j]
            if current_sum > max_sum:
                max_sum = current_sum
    return max_sum

def total_max_subarray_sum(n, initial_seed, min_val, max_val):
    total_sum = 0
    lcg_gen = lcg(initial_seed)
    for _ in range(20):
        seed = next(lcg_gen)
        total_sum += max_subarray_sum(n, seed, min_val, max_val)
    return total_sum

# Parameters
n = 10000          # Number of random numbers
initial_seed = 42  # Initial seed for the LCG
min_val = -10      # Minimum value of random numbers
max_val = 10       # Maximum value of random numbers

# Timing the function
import time
start_time = time.time()
result = total_max_subarray_sum(n, initial_seed, min_val, max_val)
end_time = time.time()

print("Total Maximum Subarray Sum (20 runs):", result)
print("Execution Time: {:.6f} seconds".format(end_time - start_time))
"""

```

lcg(seed, a, c, m): Tạo số ngẫu nhiên bằng phương pháp Linear Congruential Generator (LCG).

max_subarray_sum(n, seed, min_val, max_val): Tìm dãy con có tổng lớn nhất trong mảng số ngẫu nhiên.

total_max_subarray_sum(n, initial_seed, min_val, max_val): Chạy thuật toán 20 lần để tính tổng kết quả.

```
def stream_gpt(python):
    stream = openai.chat.completions.create(model=OPENAI_MODEL, messages=messages_for(python))
    reply = ""
    for chunk in stream:
        fragment = chunk.choices[0].delta.content or ""
        reply += fragment
        yield reply.replace('```cpp\n', '').replace('```', '')
```

Gửi mã Python đến mô hình CodeQwen và nhận về mã C++.

```
def stream_claude(python):
    result = claude.messages.stream(
        model=CLAUDE_MODEL,
        max_tokens=2000,
        system=system_message,
        messages=[{"role": "user", "content": user_prompt_for(python)}],
    )
    reply = ""
    with result as stream:
        for text in stream.text_stream:
            reply += text
            yield reply.replace('```cpp\n', '').replace('```', '')
```

- ☐ Gửi mã Python đến GPT và nhận kết quả C++ dưới dạng dòng dữ liệu (streaming).
- ☐ Xóa định dạng Markdown (```cpp) khỏi đầu ra.

```

with gr.Blocks() as ui:
    with gr.Row():
        python = gr.Textbox(label="Python code:", lines=10, value=python_hard)
        cpp = gr.Textbox(label="C++ code:", lines=10)
    with gr.Row():
        model = gr.Dropdown(["GPT", "Claude"], label="Select model", value="GPT")
        convert = gr.Button("Convert code")

    convert.click(optimize, inputs=[python, model], outputs=[cpp])

ui.launch(inbrowser=True)

```

Dùng Gradio để tạo giao diện đồ họa:

Ô nhập mã Python.

Nút chọn mô hình AI (GPT hoặc Claude).

Nút chuyển đổi mã (convert).

Hiển thị mã C++ sau khi chuyển đổi.

```

def execute_python(code):
    try:
        output = io.StringIO()
        sys.stdout = output
        exec(code)
    finally:
        sys.stdout = sys.__stdout__
    return output.getvalue()

```

Thực thi mã Python và trả về kết quả đầu ra (stdout).

```
def execute_cpp(code):
    write_output(code)
    compiler_cmd = ["clang++", "-O3", "-std=c++17", "-march=armv8.3-a", "-o", "optimized", "o
    try:
        compile_result = subprocess.run(compiler_cmd, check=True, text=True, capture_output=True)
        run_cmd = ["/optimized"]
        run_result = subprocess.run(run_cmd, check=True, text=True, capture_output=True)
        return run_result.stdout
    except subprocess.CalledProcessError as e:
        return f"An error occurred:\n{e.stderr}"
```

Biên dịch và chạy mã C++ bằng Clang++.

```
from huggingface_hub import login, InferenceClient
from transformers import AutoTokenizer

hf_token = os.environ['HF_TOKEN']
login(hf_token, add_to_git_credential=True)
```

Đăng nhập vào Hugging Face để sử dụng mô hình AI.

```
code_qwen = "Qwen/CodeQwen1.5-7B-Chat"
code_gemma = "google/codegemma-7b-it"
CODE_QWEN_URL = "https://h1vdol7jxhje3mpn.us-east-1.aws.endpoints.huggingface.cloud"
CODE_GEMMA_URL = "https://c5hggiyqachmgnqg.us-east-1.aws.endpoints.huggingface.cloud"
```

Khai báo hai mô hình AI chuyên xử lý mã nguồn (CodeQwen và CodeGemma).

```
def stream_code_qwen(python):
    tokenizer = AutoTokenizer.from_pretrained(code_qwen)
    messages = messages_for(python)
    text = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    client = InferenceClient(CODE_QWEN_URL, token=hf_token)
    stream = client.text_generation(text, stream=True, details=True, max_new_tokens=3000)
    result = ""
    for r in stream:
        result += r.token.text
    yield result
```


Gửi mã Python đến mô hình CodeQwen và nhận về mã C++.

```
def optimize(python, model):
    if model=="GPT":
        result = stream_gpt(python)
    elif model=="Claude":
        result = stream_claude(python)
    elif model=="CodeQwen":
        result = stream_code_qwen(python)
    else:
        raise ValueError("Unknown model")
    for stream_so_far in result:
        yield stream_so_far
```

Mục đích:

Chuyển đổi mã Python sang C++ bằng AI (GPT, Claude, hoặc CodeQwen).

Sử dụng generator (yield) để truyền dữ liệu theo từng phần thay vì đợi kết quả hoàn tất.

```
def select_sample_program(sample_program):
    if sample_program=="pi":
        return pi
    elif sample_program=="python_hard":
        return python_hard
    else:
        return "Type your Python program here"
```

Mục đích:

Chọn chương trình mẫu (pi hoặc python_hard) để hiển thị trong ô nhập liệu Python.

```
def run_cmd(command_to_run):
    try:
        run_result = subprocess.run(command_to_run, check=True, text=True, capture_output=True)
        return run_result.stdout if run_result.stdout else "SUCCESS"
    except:
        return ""
```

Mục đích:

Chạy lệnh hệ thống (subprocess.run).

Nếu có đầu ra (stdout), trả về kết quả, nếu không thì trả về "SUCCESS".

```
def c_compiler_cmd(filename_base):  
    my_platform = platform.system()  
    my_compiler = []
```

Mục đích:

Xác định trình biên dịch C++ phù hợp theo hệ điều hành (Windows, Linux, Mac).

```
if my_platform == "Windows":  
    if os.path.isfile(VISUAL_STUDIO_2022_TOOLS):  
        compile_cmd = ["cmd", "/c", VISUAL_STUDIO_2022_TOOLS, "&", "cl", "simple.cpp"]  
        if run_cmd(compile_cmd):  
            if run_cmd(["./simple.exe"]) == "Hello":  
                my_compiler = ["Windows", "Visual Studio 2022", ["cmd", "/c", VISUAL_STUDIO_2022_TOOLS, "&", "cl", "simple.cpp"]]
```

Mục đích:

Kiểm tra xem có thể sử dụng Visual Studio 2022 hay không bằng cách:

Biên dịch simple.cpp.

Chạy simple.exe và kiểm tra xem đầu ra có phải "Hello" không.

Nếu không có Visual Studio 2022, kiểm tra Visual Studio 2019 theo cách tương tự.

```
elif my_platform == "Linux":  
    compile_cmd = ["g++", "simple.cpp", "-o", "simple"]  
    if run_cmd(compile_cmd):  
        if run_cmd(["./simple"]) == "Hello":  
            my_compiler = ["Linux", "GCC (g++)", ["g++", f"{filename_base}.cpp", "-o", f"{filename_base}.exe"]]
```

Mục đích:

Kiểm tra và sử dụng trình biên dịch GCC (g++) trên Linux.

Nếu không có GCC, kiểm tra Clang++.

```
elif my_platform == "Darwin":
    compile_cmd = ["clang++", "-Ofast", "-std=c++17", "-march=armv8.5-a", "-mtune=apple-m1", '
    if run_cmd(compile_cmd):
        if run_cmd(["./simple"]) == "Hello":
            my_compiler = ["Macintosh", "Clang++", ["clang++", "-Ofast", "-std=c++17", "-marcl
```

Mục đích:

Kiểm tra và sử dụng Clang++ trên macOS (Darwin).

```
ui.launch(inbrowser=True)
```

Mục đích:

Khởi chạy ứng dụng web trong trình duyệt.